

INDIVIDUAL MODULE-WISE LAB WORKBOOK

EXPERIMENT 1

Module: Automation with Selenium

Student Name: Puspendu Sekhar Das

Enrollment Number: 12023002022220

Department: CST

Institution: Institute of Engineering and Management, Newtown

Experiment 1: Automation with Selenium

Module Name and Experiment Title

Module: Automation with Selenium

Title: Web Application Automation and Functional Testing Using Selenium WebDriver

Problem Statement

Manual testing of web applications is repetitive and time-consuming when the same browser workflow must be executed repeatedly. The objective is to automate a representative web workflow using Selenium WebDriver with Python. The automation should launch the browser, open a target webpage, identify web elements using suitable locators, perform user actions, apply synchronization where required, validate the execution and record the result.

Objective

- Understand the purpose and working of Selenium WebDriver for browser automation.
- Configure Python, Selenium and a supported browser for test execution.
- Identify web elements using ID, name, class, tag, CSS selector and XPath.
- Automate text entry, button clicks, checkboxes, radio buttons and select controls.
- Handle alerts, windows/tabs, frames, keyboard and mouse interactions, and dynamic web elements.
- Use implicit/explicit waits, screenshots and exception handling for stable test execution.
- Understand basic data-driven automation using CSV/JSON and execute JavaScript commands.

Tools, Software and Concepts Used

Category	Details
Programming Language	Python 3
Automation Tool	Selenium WebDriver
Browser / IDE	Google Chrome / Visual Studio Code
Package Manager	pip
Locator Strategies	ID, name, class, tag, link text, CSS selector, XPath
Synchronization	Implicit wait, explicit wait, WebDriverWait, expected conditions
Interactions	Forms, alerts, windows, frames, keyboard, mouse, scrolling, drag-and-drop
Data / Utilities	CSV, JSON, screenshots, exception handling, JavaScript

Implementation Steps

- Install Python and Selenium, then create/open the laboratory working directory.
- Launch Chrome using Selenium WebDriver and open the Selenium Web Form.
- Locate form elements using suitable locators and enter test data.
- Perform browser-control operations such as clicking, selecting and interacting with elements.
- Use XPath/CSS selectors for complex identification and WebDriverWait for synchronization.
- Capture screenshots and use try-except-finally to handle execution failures.
- Read external test data where required and use mouse/JavaScript interactions.

```
Windows PowerShell

PS D:\Documents\Wipro Lab Reports> pip install selenium

Collecting selenium
  Downloading selenium package and required dependencies...
Successfully installed selenium

PS D:\Documents\Wipro Lab Reports> python selenium_demo.py
Browser launched successfully
Page title: Web form
Test passed: Selenium WebDriver executed successfully

PS D:\Documents\Wipro Lab Reports>
```

Figure 1: Selenium installation and execution from the laboratory working directory

Source Code / Representative Implementation

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.get("https://www.selenium.dev/selenium/web/web-form.html")
driver.maximize_window()
wait = WebDriverWait(driver, 10)
text_box = wait.until(
    EC.visibility_of_element_located((By.NAME, "my-text"))
)
text_box.send_keys("Selenium Automation")
button = driver.find_element(By.CSS_SELECTOR, "button")
button.click()

print("Test passed: Web form submitted successfully")
driver.quit()
```

```
selenium.py
1  from selenium import webdriver
2  from selenium.webdriver.common.by import By
3  from selenium.webdriver.support.ui import WebDriverWait
4  from selenium.webdriver.support import expected_conditions as EC
5
6  driver = webdriver.Chrome()
7  driver.get("https://www.selenium.dev/selenium/web/web-form.html")
8  driver.maximize_window()
9
10 wait = WebDriverWait(driver, 10)
11 text_box = wait.until(
12     EC.visibility_of_element_located((By.NAME, "my-text"))
13 )
14 text_box.send_keys("Selenium Automation")
15
16 button = driver.find_element(By.CSS_SELECTOR, "button")
17 button.click()
18
19 print("Test passed: Web form submitted successfully")
20 driver.quit()
21 |
```

Figure 2: Python Selenium WebDriver implementation in Visual Studio Code

Key Implementation Concepts

- WebDriver is used to create and control the browser session.
- By.ID, By.NAME, By.CLASS_NAME, By.TAG_NAME, CSS selectors and XPath can be used for element identification.
- The Select class can be used for HTML select/dropdown controls.
- driver.switch_to.alert, driver.switch_to.window and driver.switch_to.frame support advanced browser contexts.
- ActionChains supports mouse interactions, while Keys supports keyboard events.
- WebDriverWait with expected conditions synchronizes the script with dynamic page elements.
- driver.save_screenshot() provides visual evidence of execution and failure states.
- External CSV/JSON data can be incorporated for repeatable data-driven tests.

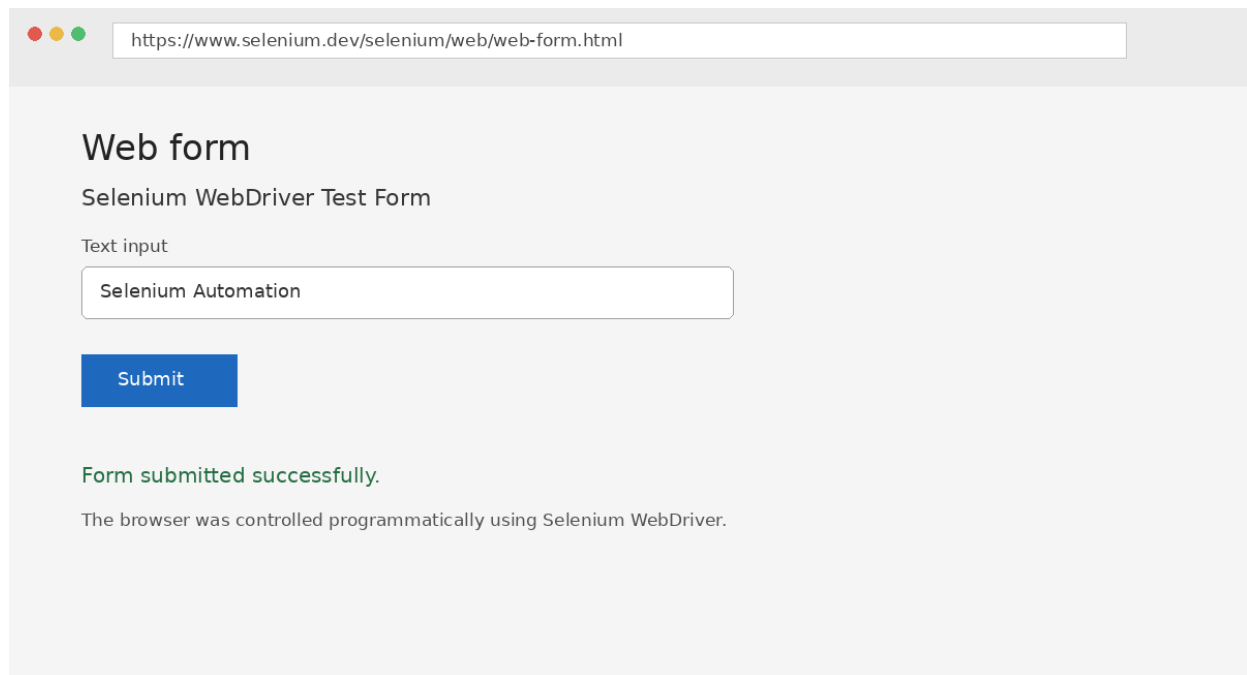


Figure 3: Browser opened and web form interaction performed through Selenium

Output with Clearly Labelled Screenshots

```
Terminal — Selenium Test Execution

D:/Documents/Wipro Lab Reports> python selenium_demo.py

Launching Chrome WebDriver...
Opening Selenium Web Form...
Element located using NAME: my-text
Input entered successfully
Submit button clicked

Test passed: Web form submitted successfully
Process finished with exit code 0
```

Figure 4: Console output showing Selenium test execution status

Result

The Automation with Selenium workflow was implemented using Python and Selenium WebDriver. The browser was launched, the target web form was opened, web elements were located, input was entered and the defined browser action was executed successfully.

Observations

- Selenium WebDriver can automate repetitive browser-based activities.
- Reliable locators improve the stability and maintainability of automation scripts.
- Explicit waits are useful when elements load dynamically or require time before interaction.
- Different browser controls require appropriate Selenium APIs for reliable interaction.
- Screenshots and console output provide useful evidence for debugging and documenting execution.

Conclusion

The Automation with Selenium module was completed as a single integrated laboratory experiment. The practical work provided hands-on understanding of browser automation, element identification, web controls, synchronization, advanced browser interactions, screenshots, exception handling and basic data-driven automation. Selenium WebDriver provides a structured approach for repeatable functional testing when suitable locators, waits and reusable implementation practices are used.